1. Speech to text (ASR, online/offline)

1. Speech to text (ASR, online/offline)

1. Concept introduction

1.1 What is "ASR"?

1.2 Implementation principle

1. **Acoustic model**
2. **Language model**
3. **Pronunciation dictionary**
4. **Decoder**
5. **End-to-end ASR**

2. Code analysis

Key code

1. Speech processing and recognition core (`largemodel/largemodel/asr.py`)
2. VAD smart recording (`largemodel/largemodel/asr.py`)

Code analysis

3. Hands-on operation

3.1 Configure the ASR

3.1.1 Configure the offline ASR

3.1.2 Configure the online ASR

3.2 Start it and test the function

---

1. Concept introduction

1.1 What is "ASR"?

ASR (Automatic Speech Recognition) is a technology that turns a human speech signal into text. It is widely used in the fields of intelligent assistants, voice command control, telephone customer service automation and real-time caption generation. The goal of ASR is to let the machine "understand" human language and convert it into a form the computer can process and understand.

1.2 Implementation principle

The implementation of an ASR system mainly relies on the following key technical components:

1. Acoustic model

- The acoustic model is responsible for converting the input audio signal into phonemes or sub-word units. This usually involves a feature extraction step, such as representing the audio signal with Mel-frequency cepstral coefficients (MFCCs) or filter banks.
- These features are then fed into a deep neural network (DNN), a convolutional neural network (CNN), a recurrent neural network (RNN) or a more advanced Transformer architecture for training, so that it learns how to map the audio features to the corresponding phonemes or text.

2. Language model

- The language model predicts the next most likely word given the preceding context, which helps to improve the recognition accuracy. It is trained on large amounts of text data and knows which word sequences are more likely to occur.
- Common language models include the n-gram model, the RNN-based LM and the recently popular Transformer-based LM.

3. Pronunciation dictionary

- The pronunciation dictionary provides the mapping between the words and their corresponding pronunciations. This is crucial for connecting the acoustic model and the language model, because it allows the system to understand and match the sounds it hears according to the known pronunciation rules.

4. Decoder

- The task of the decoder is to find the most likely sequence of words as the output, given the acoustic model, the language model and the pronunciation dictionary. This process usually involves complex search algorithms, such as the Viterbi algorithm or graph based search methods, that look for the optimal path.

5. End-to-end ASR

- With the development of deep learning, end-to-end ASR systems have appeared. They try to learn the text output directly from the raw audio signal, without an explicit separated design of the acoustic model, the pronunciation dictionary and the language model. Such systems are often based on the sequence-to-sequence (Seq2Seq) framework, for example using an attention mechanism or a Transformer architecture, which greatly simplifies the complexity of the traditional ASR systems.

In general, modern ASR systems combine the components above and are trained with large-scale datasets and powerful computing resources, so that they can turn human speech into text efficiently and accurately. As the technology advances, the performance of the ASR systems keeps improving and the application scenarios keep widening.

---

2. Code analysis

Key code

1. Speech processing and recognition core (`largemodel/largemodel/asr.py`)

```
# From largemodel/largemodel/asr.py
def kws_handler(self)->None:
    if self.stop_event.is_set():
        return

    if self.listen_for_speech(self.mic_index):
        asr_text = self.ASR_conversion(self.user_speechdir)  # Perform the ASR conversion
        if asr_text =='error':  # Check whether the length of the ASR result is less than 4
characters
```

```
            self.get_logger().warn("I still don't understand what you mean. Please try
again")
            playsound(self.audio_dict[self.error_response])  # Error response
        else:
            self.get_logger().info(asr_text)
            self.get_logger().info("okay😐, let me think for a moment...")
            self.asr_pub_result(asr_text)  # Publish the ASR result
    else:
        return

def ASR_conversion(self, input_file:str)->str:
    if self.use_oline_asr:
        result=self.modelinterface.oline_asr(input_file)
        if result[0] == 'ok' and len(result[1]) > 4:
            return result[1]
        else:
            self.get_logger().error(f'ASR Error:{result[1]}')  # ASR error.
            return 'error'
    else:
        result=self.modelinterface.SenseVoiceSmall_ASR(input_file)
        if result[0] == 'ok' and len(result[1]) > 4:
            return result[1]
        else:
            self.get_logger().error(f'ASR Error:{result[1]}')  # ASR error.
            return 'error'
```

2. VAD smart recording (`largemodel/largemodel/asr.py`)

```
# From largemodel/largemodel/asr.py
def listen_for_speech(self,mic_index=0):
    p = pyaudio.PyAudio()   # Create the PyAudio instance.
    audio_buffer = []       # Store the audio data.
    silence_counter = 0     # Silence counter.
    MAX_SILENCE_FRAMES = 90  # Stop after 900ms of silence (30 frames * 30ms)
    speaking = False  # Flag indicating the speech activity.
    frame_counter = 0  # Frame counter.
    stream_kwargs = {
        'format': pyaudio.paInt16,
        'channels': 1,
        'rate': self.sample_rate,
        'input': True,
        'frames_per_buffer': self.frame_bytes,
    }
    if mic_index != 0:
        stream_kwargs['input_device_index'] = mic_index

    # Prompt the user to speak via the buzzer.
    self.pub_beep.publish(UInt16(data = 1))
    time.sleep(0.5)
    self.pub_beep.publish(UInt16(data = 0))

    try:
```

```
        # Open the audio stream.
        stream = p.open(**stream_kwargs)
        while True:
            if self.stop_event.is_set():
                return False

            frame = stream.read(self.frame_bytes, exception_on_overflow=False)  # Read the
audio data.
            is_speech = self.vad.is_speech(frame, self.sample_rate)  # VAD detection.

            if is_speech:
                # Speech activity detected.
                speaking = True
                audio_buffer.append(frame)
                silence_counter = 0
            else:
                if speaking:
                    # Detect the silence after the speech activity.
                    silence_counter += 1
                    audio_buffer.append(frame)  # Keep recording into the buffer.

                    # End the recording when the silence duration reaches the threshold.
                    if silence_counter >= MAX_SILENCE_FRAMES:
                        break
            frame_counter += 1
            if frame_counter % 2 == 0:
                self.get_logger().info('1' if is_speech else '-')
                # Real-time status display.
    finally:
        stream.stop_stream()
        stream.close()
        p.terminate()

    # Save the valid recording (remove the trailing silence).
    if speaking and len(audio_buffer) > 0:
        # Trim the last silent part.
        clean_buffer = audio_buffer[:-MAX_SILENCE_FRAMES] if len(audio_buffer) >
MAX_SILENCE_FRAMES else audio_buffer

        with wave.open(self.user_speechdir, 'wb') as wf:
            wf.setnchannels(1)
            wf.setsampwidth(p.get_sample_size(pyaudio.paInt16))
            wf.setframerate(self.sample_rate)
            wf.writeframes(b''.join(clean_buffer))
            return True
```

Code analysis

The ASR (speech to text) function is provided by the `ASRNode` node (`asr.py`). This node is responsible for the recording, the conversion and the publishing of the audio.

1. **Audio recording (`listen_for_speech`)**:

- This function captures the audio stream from the microphone with the `pyaudio` library.
- It integrates the `webrtcvad` library for the voice activity detection (VAD). The function reads the audio frames in a loop and uses `vad.is_speech()` to decide whether each frame contains speech.
- When speech is detected, the data is written into a buffer. When a continuous silence is detected (defined by `MAX_SILENCE_FRAMES`), the recording stops.
- Finally, the audio data in the buffer is written into a `.wav` file at the path `self.user_speechdir`.

2. **Backend selection and execution (`ASR_conversion`)**:
   - The `kws_handler` function calls the `ASR_conversion` function after the recording has succeeded.
   - This function reads the ROS parameter `use_oline_asr` (a boolean) to decide which backend implementation to call.
   - If it is `false`, the `self.modelinterface.SenseVoiceSmall_ASR` method is called, which corresponds to the local recognition.
   - If it is `true`, the `self.modelinterface.oline_asr` method is called, which corresponds to the online recognition.
   - This function passes the path of the audio file as the parameter to the selected method and processes the returned result.
3. **Publishing the result (`asr_pub_result`)**:
   - After `ASR_conversion` returns valid text, `kws_handler` calls the `asr_pub_result` function.
   - This function wraps the text string in a `std_msgs.msg.String` message and publishes it to the `/asr` topic through the ROS publisher.

3. Hands-on operation

3.1 Configure the ASR

The online and the offline ASR share the microphone, the serial port, the sample rate and the language configuration, and the recognition backend is selected with `use_oline_asr`. First open the main configuration file:

```
vim ~/yahboom_ws/src/largemodel/config/yahboom.yaml
```

Keep the device settings in the existing `asr.ros__parameters` block and set the backend as needed:

```
asr:
  ros__parameters:
    use_oline_asr: False  # False: offline ASR; True: online ASR
    mic_serial_port: "/dev/ttyUSB0"
    mic_index: 0
    language: 'en'        # 'zh': Chinese; 'en': English
    regional_setting: "International"
```

First use `ls /dev/ttyUSB*` to confirm the device number of the voice module; if it is not `/dev/ttyUSB0`, change `mic_serial_port` to the actual device. If these parameters already exist, simply modify the original values and do not add duplicate keys in the same YAML parameter block.

3.1.1 Configure the offline ASR

Set `use_oline_asr` to `False` and confirm that the directory of the local SenseVoiceSmall model exists in `large_model_interface.yaml`:

```
vim ~/yahboom_ws/src/largemodel/config/large_model_interface.yaml
```

```
local_asr_model: "/home/jetson/yahboom_ws/src/largemodel/MODELS/asr/SenseVoiceSmall"
```

3.1.2 Configure the online ASR

Prepare the API Key needed by the Tongyi speech recognition service and configure the online ASR credentials and model in `large_model_interface.yaml`:

```
online_asr_api_key: "Replace it with your own online ASR API-Key"
oline_asr_model: 'paraformer-realtime-8k-v2'
oline_asr_sample_rate: 16000
```

Then set `asr.ros__parameters.use_oline_asr` in `yahboom.yaml` to `True`. The online recognition requires the Jetson to be able to reach the corresponding service, and the account has to have permission or quota available; do not publish a real API Key.

3.2 Start it and test the function

1. **Startup command**:

   ```
   ros2 launch largemodel asr_only.launch.py
   ```

2. **Test**:
   Say "Hi, yahboom" to the microphone and it replies: I'm here; you can then start talking, and at the end the audio it recorded is shown converted into text and printed in the terminal.

